```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
import pandas as pd

data = {
    'text': [
        "Free money offer",
        "Win cash prize now",
        "Limited time offer",
        "Hello how are you",
        "Let's meet tomorrow",
        "Are you coming to class",
        "Free entry in contest",
        "Call me later"
    ],
    'label': [
        1, 1, 1,   # Spam
        0, 0, 0,   # Not Spam
        1, 0
    ]
}

df = pd.DataFrame(data)
print(df)

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(df['text'])

y = df['label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
model = MultinomialNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Predictions:", y_pred)
print("Actual:", y_test.values)

print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
import joblib


joblib.dump(model, 'naive_bayes_model.pkl')


joblib.dump(vectorizer, 'vectorizer.pkl')

loaded_model = joblib.load('naive_bayes_model.pkl')


loaded_vectorizer = joblib.load('vectorizer.pkl')
text = ["Free offer now"]


text_vec = loaded_vectorizer.transform(text)


prediction = loaded_model.predict(text_vec)

print("Prediction:", prediction[0])
```

```
print("Prediction:", prediction[0])


import numpy as np

from sklearn.datasets import fetch_20newsgroups
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB


from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report

categories = ['sci.space', 'rec.sport.hockey']
data = fetch_20newsgroups(subset='all', categories=categories)

texts = data.data
labels = data.target

# Display sample
for i in range(3):
    print("Text:", texts[i][:200])
    print("Label:", labels[i])
    print("------")
import re

def preprocess(text):
    text = text.lower()  # lowercase
    text = re.sub(r'[^a-z\s]', '', text)  # remove punctuation
    return text

texts = [preprocess(t) for t in texts]
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(texts)

X_train, X_test, y_train, y_test = train_test_split(
    X, labels, test_size=0.2, random_state=42
)
model = MultinomialNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Predictions:", y_pred)
print("Actual:", y_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
import joblib


joblib.dump(model, 'naive_bayes_model_news.pkl')


joblib.dump(vectorizer, 'vectorizer_news.pkl')
# Load model
loaded_model = joblib.load('naive_bayes_model_news.pkl')

# Load vectorizer
loaded_vectorizer = joblib.load('vectorizer_news.pkl')
test_samples = [
    "NASA launched a new satellite into space",
    "The hockey team won the championship game",
    "3D rendering and computer graphics techniques improved"
]

# Transform
test_vec = loaded_vectorizer.transform(test_samples)

# Predict
predictions = loaded_model.predict(test_vec)
```

```
# Map labels to category names
for text, pred in zip(test_samples, predictions):
    print("\nText:", text)
    print("Predicted Category:", data.target_names[pred])
```

```
                       text  label
0         Free money offer      1
1       Win cash prize now      1
2       Limited time offer      1
3        Hello how are you      0
4      Let's meet tomorrow      0
5  Are you coming to class      0
6    Free entry in contest      1
7            Call me later      0
Predictions: [0 0]
Actual: [1 0]
Accuracy: 0.5

Confusion Matrix:
[[1 0]
 [1 0]]

Classification Report:
              precision    recall  f1-score   support

           0       0.50      1.00      0.67         1
           1       0.00      0.00      0.00         1

    accuracy                           0.50         2
   macro avg       0.25      0.50      0.33         2
weighted avg       0.25      0.50      0.33         2

Prediction: 1
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defi
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defi
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defi
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
Text: From: mccall@mksol.dseg.ti.com (fred j mccall 575-3539)
Subject: Re: Vandalizing the sky.
Article-I.D.: mksol.1993Apr22.204742.10671
Organization: Texas Instruments Inc
Lines: 62

In <C5tvL2.1In@herme
Label: 1
------
Text: From: epritcha@s.psych.uiuc.edu ( Evan Pritchard)
Subject: Re: div. and conf. names
Distribution: na
Organization: UIUC Department of Psychology
Lines: 59

maynard@ramsey.cs.laurentian.ca (Roger Mayna
Label: 0
------
Text: From: baalke@kelvin.jpl.nasa.gov (Ron Baalke)
Subject: Galileo Update - 04/29/93
Keywords: Galileo, JPL
Organization: Jet Propulsion Laboratory
Lines: 134
```